Prometheus Guide: Monitoring, Architecture, and Alerting System Overview

1. 什麼是 Monitoring，我們為什麼需要它？
2. 什麼是 Prometheus？
3. Monitoring 系統是怎麼運作的
4. 架構
5. 重要名詞
6. 在本機部署 Prometheus
7. Web UI
8. Alert Rules
9. Alert 是怎麼傳到 Discord 的
10. Next Step
11. Resources

---

1. 什麼是 Monitoring，我們為什麼需要它？

假設你是一個醫生，正在監測病人的生理數值——心跳、血壓、體溫。一旦某個數值超出正常範圍，你就會收到警示，在狀況惡化之前採取行動。
SRE 的 monitoring 就是在做一樣的事，我們持續檢查服務和基礎設施的健康狀態，一旦有什麼不對勁，我們會馬上收到通知，在使用者受到影響之前就先把問題解決掉，或是出問題的時候可以知道是哪裡出狀況並即時處理。

可觀測性（Observability / O11Y）的三大支柱

Data Type	功能	例子
Metrics	「有東西出問題了嗎？」	CPU 使用率飆到 95%
Logs	「發生了什麼事？」	`ERROR: database connection timeout at 14:32:05`
Traces	「問題出在哪裡？」	一個 request 花了 5 秒，其中 4.8 秒卡在 database query

這份文件的重點放在 **Metrics**——也就是我們從各個服務蒐集來的數值資料。

---

2. 什麼是 Prometheus？

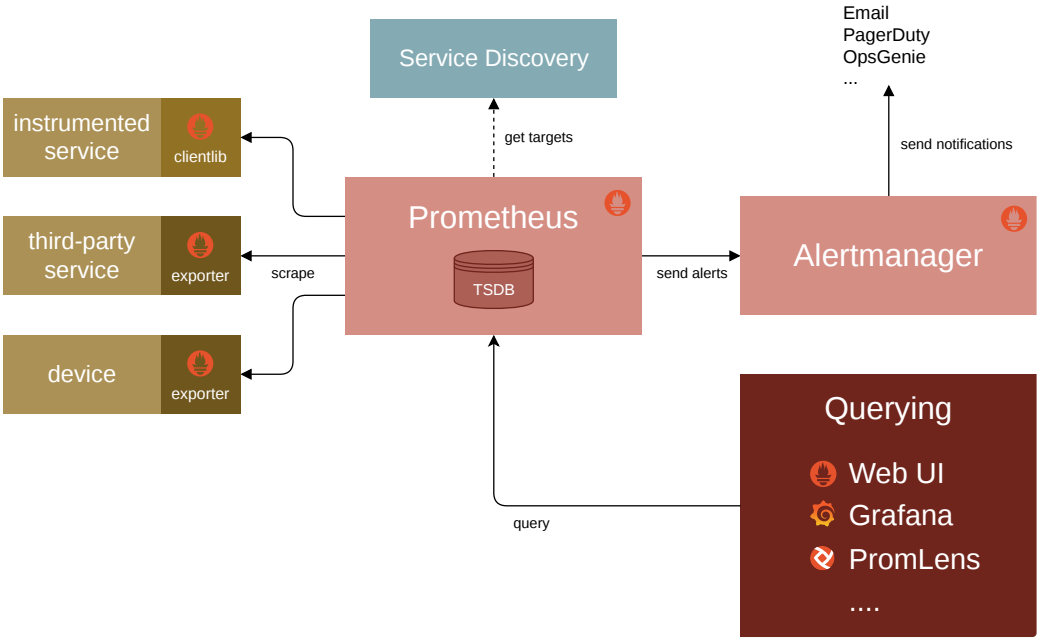
Prometheus 是一套開源的 monitoring 和 alerting 工具。它特別的地方有兩個：

Pull-based 的設計

大多數的 monitoring 系統是「被動」的，等服務主動把資料 push 過來。Prometheus 則是「主動出擊」——它會依照固定的時間間隔，主動去各個服務 pull（scrape）metrics。

```
傳統做法（Push）：  Service ── 推送資料 ──▶ Monitoring System
Prometheus（Pull）：Prometheus ──「你還好嗎？」──▶ Service
```

為什麼要用 pull？因為只要某個服務沒有回應 scrape 請求，Prometheus 就能立刻察覺它掛掉了。而且服務本身也不需要知道要把資料送到哪裡，設定更乾淨。



Overview of Prometheus System Architecture

Time Series 儲存

每一筆 metric 都是以 **time series** 的形式儲存——也就是一連串帶有時間戳記的數值：

```
http_requests_total{service="grafana", env="prod"} @ 14:00:00 = 1000
http_requests_total{service="grafana", env="prod"} @ 14:00:30 = 1042
http_requests_total{service="grafana", env="prod"} @ 14:01:00 = 1089
```

每一筆資料包含四個部分：

- **Metric 名稱**（`http_requests_total`）
- **Labels**（`service="grafana"`，`env="prod"`）——用來識別這筆資料是誰的、在哪個環境
- **數值**（`1000`）——實際量測到的數字

- **Timestamp**（`14:00:00`）——這筆資料是什麼時候記錄的

---

3. Monitoring 系統是怎麼運作的

我們的 monitoring 系統由四個主要Service組成：

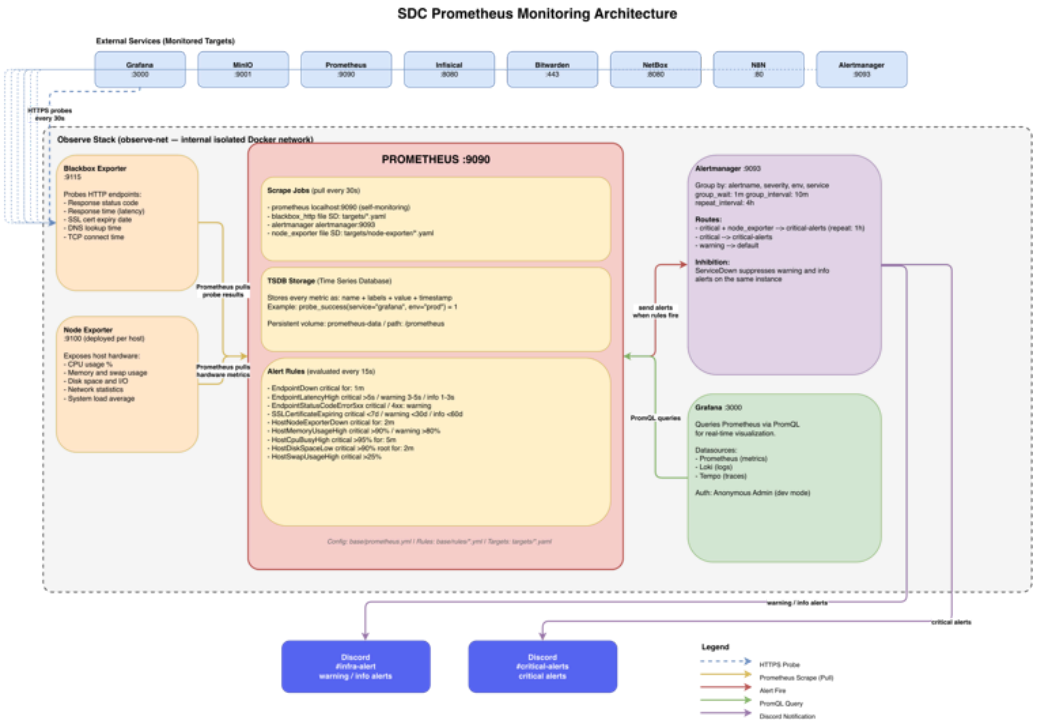
Service	負責做什麼	類比
Prometheus	蒐集並儲存 metrics、評估 alert rules	醫院的監測儀器
Blackbox Exporter	從外部探測服務是否正常（HTTP checks）	護士去確認病人有沒有回應
Alertmanager	路由並發送 alert 通知	醫院的廣播叫號系統
Grafana	把 metrics 視覺化成 dashboard	病房裡顯示生理數值的螢幕

整體流程

```
Step 1: COLLECT（蒐集）
  Prometheus 每 30 秒 scrape 一次以下來源：
  - 自己本身（self-monitoring）
  - Blackbox Exporter（它負責探測我們的 HTTP endpoints）
  - Node Exporter（回報伺服器的硬體 metrics）
  - Alertmanager（Alertmanager 自己的 metrics）

Step 2: EVALUATE（評估）
  每 15 秒，Prometheus 對照 alert rules 檢查收到的 metrics：
  「有沒有哪個服務的 probe_success == 0？」（服務掛掉了嗎？）
  「有沒有哪個服務的 probe_duration_seconds > 5？」（服務回應太慢？）

Step 3: ALERT（觸發告警）
  一旦某條 rule 被觸發，Prometheus 就把 alert 傳給 Alertmanager。
  Alertmanager 負責把相關的 alerts 分組、去重複、然後路由。

Step 4: NOTIFY（通知）
  Alertmanager 把格式化過的通知送到我們的 Discord 頻道。
  Critical alerts 進 #critical-alerts，其他的進 #infra-alert。

Step 5: VISUALIZE（視覺化）
  Grafana 查詢 Prometheus，把資料顯示成即時 dashboard。
  SRE 團隊成員可以透過 dashboard 直觀地排查問題。
```

---

4. 架構



Blockdiagram

Workflow

```
┌──────────────┐
│   Services   │
│ (HTTP endpoints) │
└──────────────┘
        │ 被 probe
        ▼
┌──────────────┐          ┌──────────────┐
│  Blackbox    │          │ Node Exporter│
│  Exporter    │          │ (硬體 metrics │
└──────────────┘          └──────────────┘
        │ 被 scrape              │ 被 scrape
        └──────────┬──────────┘
                   ▼
            ┌──────────────┐
            │   Prometheus │
            │ (TSDB 儲存)  │
            │ (Rule 評估)  │
            └──────────────┘
                │      │
      alerts    │      │   queries
                ▼      ▼
┌──────────────┐   ┌──────────────┐
│ Alertmanager │   │   Grafana    │
└──────────────┘   │ (dashboards) │
        │          └──────────────┘
        ▼
┌──────────────┐
│   Discord    │
│ (通知)       │
└──────────────┘
```

5. 重要名詞

5.1 Metric 的四種類型

類型	行為	範例	適用場景
Counter	只增不減（重啟後歸零）	`http_requests_total`	計算累計事件次數
Gauge	可增可減	`memory_usage_bytes`	記錄某個當下的數值
Histogram	把數值分配到預設的 buckets 裡	`request_duration_seconds`	分析延遲分布
Summary	在 client-side 直接計算百分位數		需要精確百分位數時

5.2 Labels

Labels 是一組 key-value pairs，用來識別特定的 time series。它們在這些地方很重要：

- **Filtering：**「只看 production 環境的 alerts」（`env="prod"`）
- **Routing：**「把 critical alerts 送到 critical Discord 頻道」（`severity="critical"`）
- **Grouping：**「把 alerts 依照服務名稱分組」（`service="grafana"`）

我們的 labels：

Label	用途	範例
`service`	哪個服務	`grafana`, `n8n`, `prometheus`
`component`	服務下的哪個子元件	`grafana`, `webhook`
`env`	哪個環境	`prod`, `dev`
`severity`	Alert 等級	`critical`, `warning`, `info`
`project`	專案分類	`sdc-core`

5.3 PromQL

PromQL 是 Prometheus 的查詢語言，以下是幾個常用的寫法：

```
# 某個服務現在有沒有在跑？
probe_success{service="grafana"}

# 某個服務的探測花了多久？
probe_duration_seconds{service="grafana"}

# 過去 5 分鐘的 HTTP request 速率
rate(http_requests_total[5m])

# 記憶體使用率（百分比）
(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100

# CPU 使用率（百分比）
(1 - avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))) * 100
```

5.4 Blackbox vs. Whitebox Monitoring

	Blackbox	Whitebox
視角	從外部（使用者的角度）	從內部（服務自己的角度）
檢查的問題	「我能不能連到這個服務？」	「這個服務內部表現得怎麼樣？」
範例	HTTP status code、response time、SSL 憑證	CPU、記憶體、request queue 深度
工具	Blackbox Exporter	Node Exporter、應用程式本身的 metrics
需要改程式碼嗎？	不需要	需要（要在程式裡 expose `/metrics` endpoint）

---

6. 在本機部署 Prometheus

事前準備

- Docker
- `make`
- Git
- Clone 這個 repo：`git clone https://github.com/NYCU-SDC/sdc-infra && cd sdc-infra`

Step 1：啟動核心基礎設施

Traefik（我們的 reverse proxy）必須比所有服務先跑起來：

```
# 以 development 模式啟動 Traefik
make core-up-dev
```

確認 Traefik 有在跑：打開瀏覽器進 http://localhost:8080/

有dashboard 就表示成功了

Step 2：啟動 Observe Stack

```
make run SERVICE="observe whoami"
```

這個指令會一次把所有 monitoring 元件都跑起來：Prometheus、Grafana、Alertmanager、Blackbox Exporter、Loki、Tempo、MinIO

```
grafana            Up 4 minutes (healthy)          3000/tcp
loki               Up 4 minutes (healthy)          0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
prometheus         Up 4 minutes (healthy)          9090/tcp
minio              Up 4 minutes                    0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
alertmanager       Up 4 minutes (healthy)          9093/tcp
blackbox-exporter  Up 4 minutes (healthy)          9115/tcp
```

等大概 30 秒讓所有服務初始化完成，然後用這個指令確認一下：

```
# 確認所有 container 都有在跑
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" | grep -E "prometheus|grafana|alertmanager|blackbox|loki|tempo|minio"
```

Step 3：打開各個 UI

在瀏覽器裡打開這些網址：

服務	網址	你會看到什麼
Prometheus	http://prometheus.localhost	查詢介面、targets、alerts
Grafana	http://grafana.localhost	Dashboards（已自動設定好 Prometheus datasource）
Alertmanager	http://alertmanager.localhost	目前觸發中的 alerts、silences
Blackbox Exporter	http://blackbox-exporter.localhost	Probe 設定

> **注意：** 如果 `*.localhost` 無法解析，可能需要在 `/etc/hosts` 加上這幾行：
>
> ```
> 127.0.0.1 prometheus.localhost grafana.localhost alertmanager.localhost blackbox-exporter.localhost minio.localhost
> ```

---

7. Web UI

7.1 Prometheus Web UI

打開 http://prometheus.localhost

Targets 頁面（`Status > Targets`）：

- 列出所有 scrape targets 和它們目前的健康狀態
- 綠色 = 正常，紅色 = 掛掉
- 你應該會看到：`prometheus`、`blackbox_http`、`alertmanager`

Graph 頁面：
試試看這幾個 query：

```
# 查看所有目前被追蹤的 metrics
up

# 查看 Prometheus 自己的記憶體使用量
process_resident_memory_bytes{job="prometheus"}

# 查看 Prometheus 已經吸收了多少筆資料
prometheus_tsdb_head_samples_appended_total
```

Alerts 頁面（`Alerts`）：

- 列出所有設定好的 alert rules
- 狀態：Inactive（綠色）、Pending（黃色）、Firing（紅色）

7.2 Grafana

打開 http://grafana.localhost

- 不需要登入（dev 模式下預設是匿名 admin）
- Prometheus 已經自動設定成 datasource 了
- 可以試試看：點 `Explore` > 選 `Prometheus` datasource > 輸入 PromQL query

7.3 Alertmanager

打開 http://alertmanager.localhost

- 顯示目前正在觸發的 alerts
- 可以建立 **silence** 來暫時靜音某些 alerts
- 可以觀察 alert grouping 的效果

---

8. Alert Rules

我們的 alert rules 放在 `services/observe/config/prometheus/base/rules/`

範例：EndpointDown

```yaml
# 檔案：rules/endpoint-availability.yml
- alert: EndpointDown
  expr: probe_success == 0 and on(instance) (probe_success{enable_alert_down="true"})
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "{{ $labels.service }} ({{ $labels.env }}) Service Unavailable"
    description: "The component has been unreachable for over 1 minute."
```

- `alert: EndpointDown` — Alert 的名稱（PascalCase，描述症狀，不描述原因）
- `expr` — PromQL 條件式。`probe_success == 0` 代表 probe 失敗了。`on(instance)` 這段確保只有 target 上有設 `enable_alert_down="true"` 的才會觸發這個 alert。
- `for: 1m` — 條件必須持續成立 1 分鐘，alert 才會真的 fire（避免短暫的閃爍就亂發通知）
- `labels.severity: critical` — 決定這個 alert 要送到哪裡（critical 的就進 critical Discord 頻道）
- `annotations` — 給人看的說明文字，用 Go template 語法。`{{ $labels.service }}` 會自動帶入 service label 的值。

Alert Rule 分類

分類	檔案	監測什麼
Endpoint Availability	`endpoint-availability.yml`	服務連不連得上？
Endpoint Latency	`endpoint-latency.yml`	服務回應夠不夠快？
Endpoint Status Code	`endpoint-status-code.yml`	服務有沒有回傳錯誤（4xx/5xx）？

SSL Certificate	`ssl-certificate.yml`	SSL 憑證快到期了嗎？
Node Availability	`node-exporter/availability.yml`	伺服器連不連得上？
Memory & CPU	`node-exporter/memory-cpu.yml`	伺服器資源快不夠用了嗎？
Disk & Swap	`node-exporter/disk-swap.yml`	伺服器硬碟快滿了嗎？

Severity 等級

等級	代表意義
`critical`	服務已經受到影響
`warning`	有潛在風險或效能下降
`info`	狀態變化或預防性事件

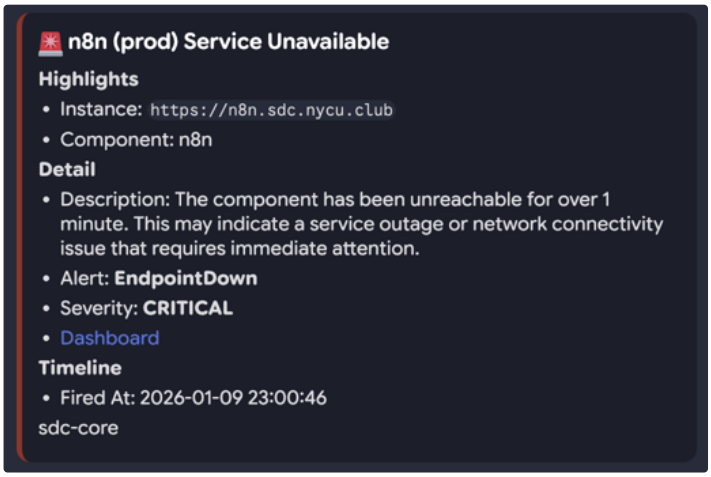
---

9. Alert 是怎麼傳到 Discord 的

Alert 的完整流程

```
1. Prometheus 每 15 秒評估一次 alert rules
         │
         ▼
2. 如果條件在設定的時間內持續成立，alert 狀態變成 FIRING
         │
         ▼
3. Prometheus 把 alert 傳給 Alertmanager (alertmanager:9093)
         │
         ▼
4. Alertmanager 依照 alertname、severity、env、service 把 alerts 分組
         │
         ▼
5. 等過了 group_wait (1 分鐘) ，Alertmanager 送出分組後的通知
         │
         ├── severity: critical + component: node_exporter
         │       └── Discord: webhook-critical (每小時重複通知)
         │
         ├── severity: critical (其他)
         │       └── Discord: webhook-critical
         │
```

```
          └── severity: warning
                  └── Discord: webhook-default
          │
          ▼
6. Discord 訊息包含：
   - 狀態 emoji (⚠️ warning、🚨 critical、✅ resolved)
   - 服務名稱和環境
   - Instance 網址
   - 問題描述
   - Severity 等級
   - 觸發時間
```



Inhibition

如果某個服務的 `ServiceDown` alert 觸發了，它會自動把同一個 instance 上所有 `warning` 和 `info` 等級的 alerts 壓下去。這樣可以避免「疲勞警告」，衍生出一堆不必要的雜訊通知。

---

10. Next Step

P0

- [ ] 打開 Prometheus 的 Targets 頁面，弄清楚 scrape target 是什麼
- [ ] 在 Prometheus Graph 頁面試著寫幾個簡單的 PromQL query
- [ ] 打開 Grafana，試著自己建一個簡單的 dashboard
- [ ] 把 `services/observe/config/prometheus/base/rules/` 下的 alert rules 看過一遍

P1

- [ ] 了解 `services/observe/config/prometheus/targets/infra.prod.yaml` 裡的 label 設計
- [ ] 研究 `services/observe/config/prometheus/base/alertmanager.yml` 裡的 routing 邏輯
- [ ] 看一下 `services/observe/config/prometheus/base/blackbox.yml` 裡有哪些 probe modules

P2

- [ ] 自己動手新增一個 monitoring target
- [ ] 寫一條自定義的 alert rule
- [ ] 為某個服務建立一個 Grafana dashboard
- [ ] 了解 GitOps 整合：monitoring 設定是怎麼自動部署的

11. Resources

- [Prometheus 官方文件]
- [PromQL Cheat Sheet]
- [Grafana 官方文件]
- [Alertmanager 官方文件]
- [Awesome Prometheus Alerts] — 現成的 alert rule 範本集

常用檔案位置

```
services/observe/
├── docker-compose.yaml                          # 所有 monitoring containers 的定義
├── config/prometheus/
│   ├── base/prometheus.yml                      # 主設定（scrape jobs、時間間隔）
│   ├── base/alertmanager.yml                    # Alert routing 到 Discord
│   ├── base/blackbox.yml                        # Probe modules（HTTP、TCP 等）
│   ├── base/rules/                              # Alert rule 定義
│   │   ├── endpoint-availability.yml
│   │   ├── endpoint-latency.yml
│   │   ├── endpoint-status-code.yml
│   │   ├── ssl-certificate.yml
│   │   └── node-exporter/
│   │       ├── availability.yml
│   │       ├── memory-cpu.yml
│   │       └── disk-swap.yml
│   ├── targets/infra.prod.yaml                  # Production 的 targets 清單
│   └── secrets/discord/                         # Webhook URLs（不放進 git）
```

常用指令

```
make run SERVICE="observe whoami"           # 啟動 monitoring stack
make stop SERVICE="observe whoami"          # 停止 monitoring stack
make logs SERVICE="observe whoami"          # 查看 logs
```

各服務的網址（Development 環境）

服務	網址
Prometheus	http://prometheus.localhost
Grafana	http://grafana.localhost
Alertmanager	http://alertmanager.localhost
Blackbox Exporter	http://blackbox-exporter.localhost
MinIO Console	http://minio.localhost